E-tiGo · Design & frontend reference

EV Charging & Vehicle Maintenance

Flows, screens, fields and API responses, as built in the backend

Prepared for	Product design (admin dashboard and driver app) and frontend / mobile engineers
Date	28 October 2026
Backend version	staging, after the EV, maintenance and repair gaps PR (28 Oct 2026, revision 2)
API base URL	`https://<staging-host>/api/v1` — every path in this guide starts here
Postman	“E-tiGo API v1” collection: Driver › EV Reservations, Repairs, Vehicle Maintenance; Admin › EV Station Management, EV Charge Requests, EV Reservations, Repair Requests, Vehicle Maintenance
Status	Everything in this guide is built. Revision 2 adds the hire-purchase fee waiver, the Operations screens for all reservations and for repair requests, and the fields listed in Part D.

Why this document exists. At Friday's presentation the client asked about drivers requesting EV charging and admins approving those requests. The PRD does not describe the screens or fields. This guide documents what the backend already does, so design and frontend can build against it without guessing. It also covers vehicle maintenance, which shares the same driver “my vehicle” area.

How to use it. Each part starts with what the feature does and the rules behind it, then goes screen by screen: what the screen is for, which endpoint feeds it, every field you can show, the actions available and the errors to handle. Example responses are at the end of each part.

Contents

Before you start 3

Part A — EV charging 4

A1. What the feature does . . . 4
A2. Statuses . . . 5
A3. Flow: request charging (reviewed) . . . 5
A4. Flow: reserve now (instant) . . . 6
A5. Driver app screens . . . 7
A6. Admin dashboard screens (Operations) . . . 10
A7. Example responses . . . 15

Part B — Vehicle maintenance 17

B1. What the feature does . . . 17
B2. Driver app screens . . . 18
B3. Admin dashboard screens (Operations) . . . 20
B4. Example responses . . . 22

Part C — Repair & service requests 23

C1. Driver side (built) . . . 23
C2. Admin dashboard screens (Operations) . . . 24

Part D — What changed in revision 2 26

Before you start

Conventions shared by every endpoint in this guide.

Topic	What to know
Auth	Every call sends `Authorization: Bearer <token>`. Driver endpoints need a driver account; admin endpoints need an admin account with the **Operations** role (Super Admin can use them too). The public station list and station detail need no login.
Times	Timestamps are ISO 8601 in UTC, e.g. `2026-10-28T13:30:00.000000Z`. Always display them in Lagos time (WAT, UTC+1). Dates without a time (`2026-11-01`) are calendar dates in Lagos.
Display strings	A few EV fields are already formatted for display and are **not** timestamps: `requested_time`, `confirmed_slot`, `check_in_time` (e.g. “02:30 pm”). Use `confirmed_slot_at` and `check_in_deadline` for any date logic.
Status fields	EV objects send status as an object: `{"value": "reserved", "label": "Reserved"}`. Use `value` in code and `label` on screen. Maintenance objects send two flat fields: `status` and `status_label`.
Money	`fee_amount` is **naira** as a string (`"500.00"`); `fee_amount_kobo` is the same in kobo (`50000`). `pricing_per_kwh` on stations is **kobo** as an integer (`15000` = ₦150.00). Prefer the kobo fields.
Lists	Paginated lists return `meta: {current_page, last_page, per_page, total}`. Pass `?page=` and, where supported, `?per_page=` (max 100).
Errors	`422` with `{message}` or `{message, errors: {field: [..]}}` for validation and refused actions; `403` for someone else's record; `404` when not found. Show `message` to the user as-is: the backend writes it in plain, user-facing language.
File uploads	Use `multipart/form-data`. Accepted: jpg, jpeg, png, pdf (repairs also accept webp), up to 10 MB.
Numbers in copy	Never hard-code the fee, minutes or days in app text. `GET /config` returns `ev_charging` (`reservation_fee_kobo, check_in_minutes, free_cancellation_minutes, max_queue_size, retry_delay_minutes`) and `vehicle_maintenance` (`due_soon_days, legal_grace_days, block_on_expired_papers`). Operations can change them.

Part A — EV charging

Drivers of electric and plug-in hybrid cars get a charging stall at an E-tiGo station. Operations run the stations and review drivers' charging requests.

A1. What the feature does

A driver can get a stall in two ways:

- **Request charging (reviewed by Operations).** The driver submits their battery level and a preferred date and time. Operations confirms a slot (and can move it to another station) or declines with a reason. When the slot starts, the system automatically holds a free stall for the driver, or puts them first in line if the station is full. This is the “request and approve” flow the client described.
- **Reserve now (instant, no review).** The driver taps Reserve on a station. The system holds a free stall at once, or queues the driver if a stall is about to free up, or tells them to try again later.

Either way the driver then drives to the station, taps **Start charging** at the stall, and taps **Finish** when done. Operations manage stations and stalls and watch occupancy live.

Who can use it

- Only drivers whose vehicle `fuel_type` is `electric` or `plug_in_hybrid`. The vehicle object has `can_charge: true|false`; hide or disable the EV entry point when it is false. Other drivers get a 422: “EV charging is only available for electric and plug-in hybrid vehicles. Set your vehicle's fuel type if it is one.”
- One open request or reservation per driver at a time (pending, confirmed, reserved, queued or active). A second attempt gets 422 “You already have an active reservation.”
- Only stations with status `active` take reservations and appear in the driver app.

Who pays the reservation fee

Driver	Pays ₦500?	fee_waiver_reason
On an E-tiGo hire-purchase (fleet) agreement that is **active** or **paused**: still paying the daily ₦40,000 remittance for the car	No	`hire_purchase`
Hire purchase **completed** (car paid off)	Yes	`null`
Agreement **terminated**	Yes	`null`
External driver with their own car (no agreement)	Yes	`null`
Driver whose tier waives the fee (Platinum and Diamond by default)	No	`tier`

The rule is checked when the stall is reserved or the request is confirmed, and the result is saved on the reservation. To show the fee **before** the driver reserves, read `charging.reservation_fee` from `GET /driver/vehicle/status` (Part A, Screen 1). Suggested copy for a waived fee: “Free while you're paying off your E-tiGo car.”

Rules and numbers

Rule	Value	What it means on screen
Reservation fee	₦500	Taken from the driver's **earnings** balance only when a stall is actually held for them (not when they submit a request and not while queued). It may take the balance below zero, down to the wallet's negative limit. If it can't be paid, the hold is cancelled.
Fee waiver	hire purchase or tier	Drivers still paying off an E-tiGo car, and Platinum and Diamond tiers, charge free (see above). `fee_waived: true`, `fee_amount_kobo: 0`.
Check-in window	30 minutes	After a stall is held, the driver must tap Start within 30 min or the hold expires and the fee is kept. For confirmed requests, `check_in_time` = slot time + 30 min.

Rule	Value	What it means on screen
Free cancellation	2 minutes	Cancelling within 2 min of the fee being taken refunds it. Use `free_cancellation_until` to show a countdown.
Queue	max 10 per station	Instant reservations queue only if a busy stall is due to free within 15 min; otherwise the driver is told to retry in 15 min.
Unreviewed requests	end of requested day	A request still pending when its requested date has passed (Lagos time) expires and the driver is told.
Background checks	every 1–5 min	Slot starts are checked every minute; missed check-ins every 5 minutes. Screens should refetch on push or focus.

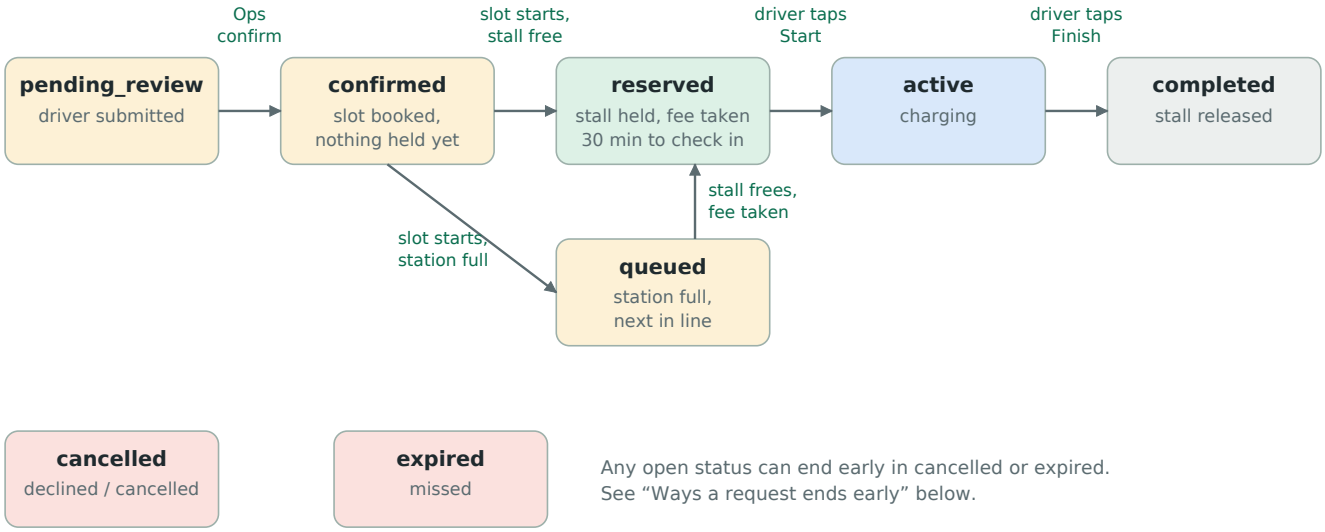
Not client-confirmed. The 30-minute check-in, 2-minute free cancellation and queue limits are engineering defaults, set in config. Only the hire-purchase waiver came from the client (28 Oct 2026). Counting a paused agreement as still paying is an engineering pick.

A2. Statuses

Every request and reservation is one record (an “EV reservation”) that moves through these statuses. Design one visual state for each.

value	label	Meaning	What the driver can do
`pending_review`	Pending Review	Request submitted; waiting for Operations.	Cancel
`confirmed`	Confirmed	Operations booked a slot. No stall is held and no fee taken yet.	See slot, station, check-in-by time. Cancel.
`reserved`	Reserved	A stall is held for the driver (`assigned_bay`); fee taken.	Navigate, **Start charging**, Cancel (refund within 2 min).
`queued`	Queued	Station full; driver is in line for the next stall.	See position and estimate. Cancel.
`active`	Active	Charging.	**Finish**
`completed`	Completed	Done; stall released.	—
`expired`	Expired	Missed: not reviewed in time, no check-in within 30 min, or the slot could not be held in time.	— (reserve again)
`cancelled`	Cancelled	Driver cancelled, Operations declined (`decline_reason`) or cancelled it (`end_reason`), the fee couldn't be paid, or the station closed (reason in `notes`).	— (reserve again)

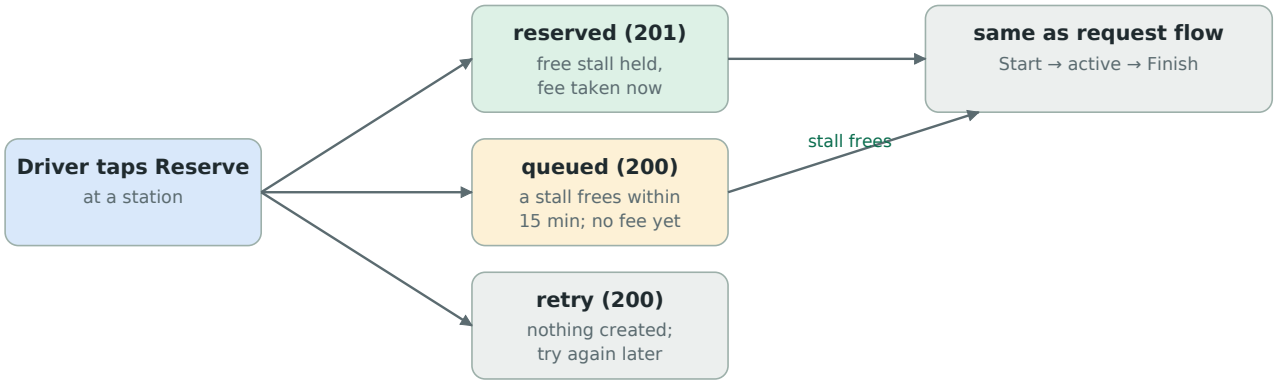
A3. Flow: request charging (reviewed)



Ways a request ends early

When	Result	Driver is told (push)
Operations decline it (while pending, or confirmed before the slot starts)	`cancelled` + `decline_reason`	“Charging Request Declined”: We couldn't book your charging request: reason. You can reserve a stall from the station list.
Nobody reviews it before the requested day ends	`expired`	“Charging Request Declined”: ...was not booked in time...
Driver cancels (any open status)	`cancelled`	— (fee refunded only within 2 min of being taken)
At slot start the station isn't active	`cancelled`, reason added to `notes`	“EV Reservation Cancelled”
At slot start the earnings balance can't cover the fee	`cancelled`	“EV Reservation Cancelled”
Slot found more than 30 min after it started	`expired`	“EV Reservation Cancelled”
Stall held but the driver doesn't tap Start within 30 min	`expired`; fee kept; stall goes to the next in line	—
Queued, a stall frees, but the fee can't be paid	`cancelled`; next driver in line is served	“EV Reservation Cancelled”
Operations cancel it from the dashboard (any open status), with or without a fee refund	`cancelled` + `end_reason`, `ended_by_operations: true`	“EV Reservation Cancelled”: ...cancelled by E-tiGo Operations: reason. [fee refunded]
Operations end a session the driver didn't finish (reserved or active)	`completed` + `end_reason`; fee kept	“Charging Session Ended”

A4. Flow: reserve now (instant)



The reserve call returns one of three outcomes in `outcome`. Only `reserved` and `queued` create a record. Instant reservations have no requested date, so they **do not** appear in the Operations charge-request queue; Operations see them in the All reservations screen (Screen F) and on the station detail screen.

A5. Driver app screens

Screen 1 — Charging card on “My vehicle”

A summary card on the vehicle screen: battery level and the current reservation, with a button into charging.

`GET` `/driver/vehicle/status` driver

Use the `charging` block of the response (it is `null` for cars that can't charge):

Field	Type	Notes
`charging.battery_level_percent`	int 0–100 | null	Last level the driver reported when going online.
`charging.battery_level_at`	timestamp | null	When it was reported. Show “updated 2 h ago”.
`charging.active_reservation`	object | null	The open request or reservation, if any.
`…active_reservation.id / status`	uuid / string	Status is a plain string here (not the value/label object).
`…active_reservation.station`	{id, name} | null	
`…active_reservation.confirmed_slot`	string | null	Display time, e.g. “02:30 pm”, once confirmed.
`charging.reservation_fee`	object	`{amount_kobo, waived, waiver_reason}`: what reserving a stall costs this driver now. Show “₦500 reservation fee” or “Free while you're paying off your E-tiGo car”.

Battery level when going online

`POST` `/driver/toggle-online` driver

Optional body field `battery_level_percent` (integer 0–100). Stored only for plug-in cars. Design a quick slider or input in the go-online step for EV drivers.

Screen 2 — Station list / map

`GET` `/ev-stations?city_id={uuid}` public, no login

`GET` `/ev-stations/{station_id}` public, no login

`city_id` is required (use the driver's city). Only active stations are listed, sorted by name. The detail call adds `stalls[]` and `city`.

Station field	Type	Notes
`id, name, address`	string	
`lat, lng`	decimal string	Map pin.
`city_id`	uuid	
`total_stalls`	int	
`connector_types`	string[]	Any of CCS2, CHAdeMO, Type2, GBT, Tesla. Show as chips.
`power_output_kw`	int | null	e.g. 60 kW.
`pricing_per_kwh`	int | null	**Kobo** per kWh: divide by 100 for naira. Informational; the app does not charge per kWh.
`operating_hours`	{open, close} | null	“HH:mm” strings, e.g. 07:00–22:00.
`status`	{value, label}	Always active in the driver list.
`availability`	object	`{total, available, occupied, reserved, out_of_service}` stall counts. Show “3 of 6 free”.
`stalls[]`	array	Detail only. Each: `id, stall_number, status{value,label}, occupied_since, estimated_departure_at`.

Design tip. Disable “Reserve now” when `availability.available` is 0 and offer “Request a slot” instead, or let the driver try: the API answers with queued or retry.

Screen 3 — Request charging form

`POST` `/driver/ev-reservations` driver

Field	Required	Rules	UI suggestion
`battery_level`	yes	integer 0–100	Slider with %; prefill from the vehicle's last battery level.
`requested_date`	no	date, today or later (Lagos). Default: today	Date picker.
`requested_time`	no	text, max 50. Default: now as “hh:mm am”	Time picker that sends “02:30 pm”. Operations read it as a preference.
`station_id`	no	uuid of a station	Station picker from Screen 2. If empty, the first active station in the driver's city is used.
`notes`	no	text, max 1000	“Anything Operations should know?”
`estimated_charge_minutes`	no	integer 5–480	“How long will you charge?” (e.g. 30 / 45 / 60 / 90 min chips). Lets drivers behind them queue instead of being told to retry.

Success (201): `{message: "Charge request submitted successfully.", reservation: {…}}` with status `pending_review`. **Errors (422):** can't charge; already has an open reservation; field validation.

Screen 4 — Reserve now

`POST` `/driver/ev-reservations/stations/{station_id}/reserve` driver

Optional body `estimated_charge_minutes` (integer 5–480). Ask “How long will you charge?”: this sets the stall's expected departure, which lets other drivers queue instead of being told to retry.

outcome	HTTP	Body	Show
`reserved`	201	`message, outcome, reservation`	Stall number, check-in countdown (30 min), fee, cancel option.
`queued`	200	`message, outcome, reservation, queue_position, estimated_available_at`	“You're #2 in line, stall expected about 14:20”. No fee yet.
`retry`	200	`message, outcome, retry_after_minutes, estimated_wait_minutes`	Nothing was created. “All stalls busy. Try again in 15 min.”

Errors (422): “This station is not currently operational.”; can't charge; already active; “Your earnings balance cannot cover the ₦500.00 reservation fee.”; “Your earnings account is on hold, so the reservation fee cannot be paid. Contact support.”

Screen 5 — My charging (list and detail)

`GET` `/driver/ev-reservations?status={value}&per_page=20` driver

`GET` `/driver/ev-reservations/{id}` driver

Newest first. The detail screen should be driven by `status.value`. The reservation object (also used on admin screens):

Field	Type	Show when / notes
`id`	uuid	
`type`	string	Always “charging”.
`status`	{value, label}	Always.

Field	Type	Show when / notes
`station_id, station_name, station_address`	string	Always (station may be null only in odd data).
`station`	object	Full station (see Screen 2) when loaded.
`stall_id, assigned_bay`	string | null	`assigned_bay` e.g. “Bay 3 · DC fast charger”. reserved / active.
`battery_level`	int | null	Requests only.
`requested_date, requested_time`	date / display string	Requests only. What the driver asked for.
`notes`	string | null	Driver's note; system reasons for a cancellation are appended here.
`confirmed_slot_at`	timestamp | null	confirmed and later. Use for countdowns.
`confirmed_slot, check_in_time`	display string | null	“Confirmed for 02:30 pm, check in by 03:00 pm”.
`approver_name`	string | null	Always “E-tiGo Operations” once reviewed (staff names are never shown to drivers).
`reviewed_at`	timestamp | null	When Operations confirmed or declined.
`decline_reason`	string | null	Declined requests. Show prominently.
`queue_position, estimated_available_at`	int / timestamp | null	queued.
`kind`	string	`request` (reviewed by Operations) or `instant` (reserved from the station list).
`estimated_charge_minutes`	int | null	The driver's estimate.
`fee_amount, fee_amount_kobo, fee_waived`	"500.00" / int / bool	Use the kobo field.
`fee_waiver_reason`	string | null	`hire_purchase` → “Free while you're paying off your E-tiGo car”; `tier` → “Free for your tier”.
`check_in_deadline`	timestamp | null	When the held stall is released if charging hasn't started (confirmed or reserved). Use for the countdown.
`ended_by_operations, end_reason, ended_at`	bool / string / timestamp	Operations cancelled or ended it. Show the reason.
`fee_charged_at, fee_refunded_at`	timestamp | null	Receipt lines.
`free_cancellation_until`	timestamp | null	Countdown for “Cancel for free”; null when there is nothing to refund.
`reserved_at, activated_at, completed_at`	timestamp | null	Timeline.
`vehicle_name, vehicle_plate, vehicle_color`	string | null	The driver's car.
`submitted_at, created_at, updated_at`	timestamp	
`queue_ahead_count, queue_ahead_description`	int / string | null	While queued: drivers ahead, and ready copy: “You're next in line”, “2 drivers ahead of you”.

Screen 6 — At the station: start, finish, cancel

POST	`/driver/ev-reservations/{id}/activate`	driver
POST	`/driver/ev-reservations/{id}/complete`	driver
POST	`/driver/ev-reservations/{id}/cancel`	driver

Action	Allowed from	Result	Refused (422) message
Start charging	`reserved`	`active`	Only reserved reservations can be activated.
Finish	`active`, `reserved`	`completed`; stall passes to the next in line	Only active or reserved reservations can be completed.
Cancel	any open status	`cancelled`; fee refunded if within `free_cancellation_until`	Cannot cancel a terminal reservation.

Each returns `{message, reservation}`. Start accepts an optional `estimated_charge_minutes` (5–480) that replaces the earlier estimate; ask for it if the driver didn't give one. Finish and Cancel take no body. All three return 403 for another driver's reservation.

Driver notifications (push + in-app inbox)

Category `ev_charging`. Every push carries `data.type`, `data.reservation_id` and `data.station_name`; deep-link to Screen 5 detail.

data.type	Title	Sent when	Example body
`ev_charge_request_confirmed`	Charging Request Confirmed	Operations confirm	Your charging slot at Lekki Phase 1 Hub is confirmed for today at 2:30 PM. We'll hold a stall from then; check in by 03:00 pm.
`ev_charge_request_declined`	Charging Request Declined	Operations decline, or nobody reviewed it in time	We couldn't book your charging request: no stalls free this afternoon. You can reserve a stall from the station list.
`ev_reservation_ready`	EV Reservation Ready / EV Stall Ready	A stall is held (slot started, or the queue reached them). Has `data.stall_number`.	Stall 3 at Lekki Phase 1 Hub is ready for you. Check in by 03:00 pm.
`ev_reservation_cancelled`	EV Reservation Cancelled	Slot or queued turn could not be held, or Operations cancelled it	Your charging reservation at Lekki Phase 1 Hub was cancelled by E-tiGo Operations: Charger 1 is faulty. Your ₦500.00 reservation fee has been refunded.
`ev_reservation_ended`	Charging Session Ended	Operations ended a session the driver didn't finish	Your charging session at Lekki Phase 1 Hub was ended by E-tiGo Operations: Car left the bay.

Live updates in the driver app

Subscribe to the driver's private channel `driver.{userId}` (already used for ride offers) and listen for `ev.reservation.updated`. It fires whenever one of the driver's requests or reservations is created or changes status, queue position, stall, station or slot. Payload: `{reservation_id, deleted, reservation}`, where `reservation` is the same object as Screen 5. Replace the screen's copy with it; no refetch needed.

A6. Admin dashboard screens (Operations)

Screen A — Charge requests queue (the approval screen)

The screen the client asked for: drivers who have requested charging, waiting for Operations to confirm or decline.

`GET` `/admin/ev-charge-requests` Operations

Query param	Notes
`status`	Comma list of status values. Default `pending_review,confirmed`. Use tabs, e.g. Pending | Confirmed | History (`reserved,queued,active,completed,cancelled,expired`). Unknown value → 422.

Query param	Notes
`station_id`	Filter by station (dropdown from Screen B).
`city_id`	Filter by the station's city.
`search`	Driver first name, last name or phone (max 100).
`page, per_page`	Default 20, max 100.

Response: `charge_requests[]` (reservation objects, see Screen 5 table, plus `driver` and `reviewed_by`), `summary: {pending_review, confirmed}` (counts for tab badges, ignore filters), and `meta`. Order: pending requests first, oldest first; then confirmed, soonest slot first.

Suggested columns

Column	Source field(s)
Driver	`driver.first_name` + `driver.last_name`, `driver.phone`, avatar `driver.avatar_url`
Vehicle	`vehicle_name`, `vehicle_plate`, `vehicle_color`
Battery	`battery_level` % (highlight low levels, e.g. under 20%)
Wanted	`requested_date` + `requested_time`
Station	`station_name` (`station_address`)
Waiting	time since `submitted_at`
Status	`status.label`
Slot (confirmed)	`confirmed_slot_at` (local), `check_in_time`
Reviewed by	`reviewed_by.name`, `reviewed_at` (admins only)
Notes / decline reason	`notes`, `decline_reason`

Action: Confirm

POST `/admin/ev-charge-requests/{id}/confirm` Operations

Field	Required	Notes
`confirmed_slot_at`	yes	Date + time of the slot. Send ISO 8601 with the Lagos offset, e.g. `2026-10-28T14:30:00+01:00`. Prefill from the driver's requested date and time. At most 5 minutes in the past and 7 days ahead; a slot that has already started holds a stall at once.
`station_id`	no	Move the request to another station. Must be active. Default: the station the driver picked.

Only `pending_review` requests can be confirmed. The driver is notified. The response is `{message, charge_request, reservation}`: the same object under both keys.

Action: Decline

POST `/admin/ev-charge-requests/{id}/decline` Operations

`reason` is required (5–500 characters) and **is shown to the driver**, so label the field accordingly. Allowed while `pending_review`, or `confirmed` before the slot starts.

When two admins act at once

A refused confirm or decline returns 422 `{message, error_code: "VALIDATION_ERROR", current_status}`. Show the message, update the row's status from `current_status` and refresh the list.

Screen B — Stations list

`GET` `/admin/ev-stations?status=&search=&city_id=&page=` Operations

Station objects as in the driver app, plus for admins: `availability.queue_length` (drivers waiting), `city`, and `stalls[]` with `current_driver` (`{id, name, phone, vehicle{make, model, colour, plate_number}}`). Station `status`: `active` (Active), `inactive` (Inactive), `maintenance` (Under Maintenance).

Screen C — Create / edit station

`POST` `/admin/ev-stations` Operations

`PUT` `/admin/ev-stations/{id}` Operations (send only changed fields)

Field	Create	Rules / UI
`name`	required	max 255
`lat, lng`	required	Map pin picker.
`address`	required	max 500
`city_id`	optional	Worked out from the pin when empty; 422 if the pin is outside every active city.
`total_stalls`	required	1–100. Stalls numbered 1..n are created. On edit, increasing adds stalls; decreasing removes the highest-numbered ones only if they are free (else 422 “Stall 5 is in use...”).
`connector_types`	optional	Multi-select: CCS2, CHAdeMO, Type2, GBT, Tesla.
`power_output_kw`	optional	1–500
`pricing_per_kwh`	optional	Integer **kobo**. Let admins type naira and multiply by 100.
`operating_hours`	optional	`{open: "07:00", close: "22:00"}` (24-hour HH:mm, both required if sent).
`status`	optional	`active | inactive | maintenance`. Default active.

Screen D — Station detail (stalls and queue)

`GET` `/admin/ev-stations/{id}` Operations

Returns `station` (with stalls and their current driver), and `queue[]`: drivers waiting for a stall, in order. Ignore the top-level `availability` (deprecated duplicate of `station.availability`).

queue[] field	Notes
`reservation_id, position`	Position 1 is next.
`driver`	`{id, name, phone, vehicle{make, model, colour, plate_number}}`
`battery_level`	Only for drivers who came from a charge request.
`joined_at, wait_seconds`	Show “waiting 12 min”.
`estimated_available_at`	When a stall is expected to free up, if known.

Stall grid: one tile per stall with `stall_number`, status colour (Available, Occupied, Reserved, Out of Service), `current_driver`, `occupied_since` and `estimated_departure_at`.

Actions

`PATCH` `/admin/ev-stations/{id}/status` body {status: active|inactive|maintenance}

`PUT` `/admin/ev-stations/{id}/stalls` body {stalls: [{stall_number, status}]}

`DELETE` `/admin/ev-stations/{id}` Operations

- **Stalls:** set any stall to available, occupied (e.g. a non-E-tiGo car), reserved or out_of_service. A new `stall_number` adds a stall. A stall holding a driver's reservation can't be changed: 422 “Stall 2 has a driver's reservation. Change it once the driver finishes or cancels.” Show those stalls as locked.

- **Status:** stations that aren't active disappear from the driver app, and confirmed slots there are cancelled when they start.
- **Delete:** refused with 422 while the station has open reservations. Ask for confirmation first.

Screen E — Utilisation dashboard

`GET` `/admin/ev-stations/utilisation?city_id=` Operations

Active stations only. `utilisation[]`: `station_id, station_name, city_id, total_stalls, availability, occupancy_rate` (% of stalls occupied or reserved), `estimated_wait_minutes` (until the soonest expected departure, or null). `summary`: `total_stations, total_stalls, total_available, total_occupied, average_occupancy_rate`.

Screen F — All reservations (search, history, cancel, end)

Every request and reservation, including the reserve-now ones that never reach the approval queue. Use it for search, history, and for fixing a reservation that is stuck.

`GET` `/admin/ev-reservations` Operations

Query param	Notes
`status`	Comma list of statuses. Default: all.
`kind`	`request` or `instant`. Show as a column or chip too.
`station_id, city_id, driver_id`	Narrow the list. `driver_id` is the driver's user id (handy from the driver profile).
`search`	Driver name, phone or vehicle plate.
`from, to`	Lagos dates, on `reserved_at`.

Response: `reservations[]` (newest first), `summary` (open reservations now by status, for KPI tiles: pending_review, confirmed, reserved, queued, active) and `meta`. Detail: `GET /admin/ev-reservations/{id}`. Show `ended_by.name` and `end_reason` when Operations acted.

Action: Cancel (any open reservation)

`POST` `/admin/ev-reservations/{id}/cancel` body {reason, refund_fee}

`reason` is required (5–500) and shown to the driver. `refund_fee` (checkbox, default off) gives the fee back: tick it when it wasn't the driver's fault, e.g. a broken charger. The stall goes to the next driver in line. The response adds `fee_refunded` (true or false).

Action: End session (reserved or active)

`POST` `/admin/ev-reservations/{id}/end` body {reason?, refund_fee?}

For a driver who drove off without tapping Finish. Marks it completed and frees the stall. The fee is kept unless `refund_fee` is ticked (e.g. the charger failed mid-session). Offer it on the station's stall tile too (the tile's `current_driver` links to the reservation through this list, filtered by `station_id` and `status=active,reserved`).

Refused (422): `{message, error_code, current_status}`, e.g. “This reservation has already ended.” Refresh the row.

Screen G — EV charging settings

`GET` `/admin/settings/ev-charging` Operations

`PUT` `/admin/settings/ev-charging` send only changed fields

Setting	Default	Control
`reservation_fee`	500 (naira)	Currency input, 0–100,000

Setting	Default	Control
`reservation_expiry_minutes`	30	Minutes a held stall waits for the driver (5–240)
`free_cancellation_minutes`	2	0–60
`max_queue_size`	10	Drivers per station (0–100)
`imminent_departure_minutes`	15	A full station queues only if a stall frees within this (0–240)
`retry_delay_minutes`	15	“Try again in” (1–240)
`max_active_reservations_per_driver`	1	1–5
`hire_purchase_fee_waiver`	on	Toggle: drivers still paying off an E-tiGo car reserve for free
`waive_fee_while_agreement_paused`	on	Toggle: a paused agreement still counts as paying

Changes apply to reservations made or confirmed afterwards; a reservation keeps the fee it was given. The maintenance settings already have their own screen (Part B, Screen E).

Live updates (admin web)

Subscribe to the private channel `admin.ev-stations` (Operations and Super Admin) and listen for `ev.station.availability_changed`. It fires after any stall or reservation change at a station:

```
{
  "station_id": "9d4c1f6e-...",
  "deleted": false,
  "status": "active",
  "availability": {"total": 6, "available": 2, "occupied": 3, "reserved": 1, "out_of_service": 0,
      "queue_length": 1},
  "updated_at": "2026-10-28T13:31:04.000000Z"
}
```

Update counts in place. When the station's detail or the charge-request queue is open, refetch it. `deleted: true` means the station was removed. The driver app has no websocket for EV; it relies on push notifications and refetching.

> **Note.** City launch readiness includes the gate “Charging stations live for plug-in drivers”, so the city checklist screen may link to the stations list.

A7. Example responses

POST /driver/ev-reservations → 201


```
{
  "message": "Charge request submitted successfully.",
  "reservation": {
    "id": "a1f0c2d4-5b6e-4f70-8a91-0b2c3d4e5f60",
    "type": "charging",
    "station_id": "9d4c1f6e-2a3b-4c5d-8e9f-0a1b2c3d4e5f",
    "stall_id": null,
    "status": {"value": "pending_review", "label": "Pending Review"},
    "battery_level": 18,
    "requested_date": "2026-10-28",
    "requested_time": "02:30 pm",
    "notes": "Need about an hour",
    "kind": "request",
    "estimated_charge_minutes": 60,
    "approver_name": null,
    "confirmed_slot": null,
    "check_in_time": null,
    "confirmed_slot_at": null,
    "reviewed_at": null,
    "decline_reason": null,
    "queue_position": null,
    "queue_ahead_count": null,
    "queue_ahead_description": null,
    "estimated_available_at": null,
    "check_in_deadline": null,
    "ended_by_operations": false,
    "end_reason": null,
    "ended_at": null,
    "fee_amount": null,
    "fee_amount_kobo": null,
    "fee_waived": false,
    "fee_waiver_reason": null,
    "fee_charged_at": null,
    "fee_refunded_at": null,
    "free_cancellation_until": null,
    "reserved_at": "2026-10-28T11:02:10.000000Z",
    "activated_at": null,
    "completed_at": null,
    "vehicle_name": "BYD Atto 3",
    "vehicle_plate": "LND-482-KJ",
    "vehicle_color": "White",
    "station_name": "Lekki Phase 1 Hub",
    "station_address": "12 Admiralty Way, Lekki Phase 1, Lagos",
    "assigned_bay": null,
    "station": { "...": "station object" },
    "driver": { "...": "user object" },
    "submitted_at": "2026-10-28T11:02:10.000000Z",
    "updated_at": "2026-10-28T11:02:10.000000Z",
    "created_at": "2026-10-28T11:02:10.000000Z"
  }
}
```

GET /admin/ev-charge-requests (trimmed)


```
{
  "charge_requests": [
    {
      "id": "a1f0c2d4-...",
      "status": {"value": "confirmed", "label": "Confirmed"},
      "battery_level": 18,
      "requested_date": "2026-10-28",
      "requested_time": "02:30 pm",
      "confirmed_slot_at": "2026-10-28T13:30:00.000000Z",
      "confirmed_slot": "02:30 pm",
      "check_in_time": "03:00 pm",
      "approver_name": "E-tiGo Operations",
      "reviewed_at": "2026-10-28T11:20:44.000000Z",
      "reviewed_by": {"id": "5e6f...", "name": "Ada Okafor"},
      "fee_amount": "500.00",
      "fee_waived": false,
      "station_name": "Lekki Phase 1 Hub",
      "vehicle_name": "BYD Atto 3",
      "vehicle_plate": "LND-482-KJ",
      "driver": {"id": "7c8d...", "first_name": "Tunde", "last_name": "Bello", "phone": "+2348012345678",
      "...": "..."},
      "...": "all other reservation fields"
    }
  ],
  "summary": {"pending_review": 4, "confirmed": 2},
  "meta": {"current_page": 1, "last_page": 1, "per_page": 20, "total": 6}
}
```


POST /driver/ev-reservations/stations/{id}/reserve → queued, and retry


```
{
  "message": "Added to queue.",
  "outcome": "queued",
  "reservation": { "status": {"value": "queued", "label": "Queued"}, "queue_position": 2, "...": "..." },
  "queue_position": 2,
  "estimated_available_at": "2026-10-28T13:20:00.000000Z"
}

{
  "message": "All stalls are occupied with no imminent departures. Please try again later.",
  "outcome": "retry",
  "retry_after_minutes": 15,
  "estimated_wait_minutes": 40
}
```


GET /admin/ev-stations/{id} (trimmed)


```
{
  "station": {
    "id": "9d4c1f6e-...", "name": "Lekki Phase 1 Hub", "city_id": "1b2c...",
    "lat": "6.4474000", "lng": "3.4723000", "address": "12 Admiralty Way, Lekki Phase 1, Lagos",
    "total_stalls": 4, "connector_types": ["CCS2", "Type2"], "power_output_kw": 60,
    "pricing_per_kwh": 15000, "operating_hours": {"open": "07:00", "close": "22:00"},
    "status": {"value": "active", "label": "Active"},
    "availability": {"total": 4, "available": 1, "occupied": 2, "reserved": 1, "out_of_service": 0,
      "queue_length": 1},
    "city": {"id": "1b2c...", "name": "Lagos", "...": "..."},
    "stalls": [
      {"id": "s1...", "stall_number": 1, "status": {"value": "occupied", "label": "Occupied"},
       "occupied_since": "2026-10-28T12:40:00.000000Z", "estimated_departure_at":
      "2026-10-28T13:40:00.000000Z",
       "current_driver": {"id": "7c8d...", "name": "Tunde Bello", "phone": "+2348012345678",
                          "vehicle": {"make": "BYD", "model": "Atto 3", "colour": "White", "plate_number":
      "LND-482-KJ"}},
       "updated_at": "2026-10-28T12:40:00.000000Z"}
    ]
  },
  "availability": { "...": "deprecated, same as station.availability" },
  "queue": [
    {"reservation_id": "q1...", "position": 1,
     "driver": {"id": "8a9b...", "name": "Kemi Ade", "phone": "+2348098765432", "vehicle": {"...": "..."}},
     "battery_level": null, "joined_at": "2026-10-28T13:05:00.000000Z", "wait_seconds": 720,
     "estimated_available_at": "2026-10-28T13:40:00.000000Z"}
  ]
}
```

Part B — Vehicle maintenance

Every vehicle has a checklist of service items and legal papers. The system works out what is due, reminds drivers, and keeps a car off the road when its papers have expired.

B1. What the feature does

- Each vehicle gets a list of **items**. Service items are due by distance or time; legal papers are due on their expiry date.
- Drivers see their car's health, get reminders as items become due, and log work they've had done (with a receipt) for Operations to approve.
- Fleet (E-tiGo-owned) cars are maintained by Operations: drivers can't log work on them, and Operations record the work and upload the papers.
- When a legal paper has been expired for longer than the grace period, the driver **can't go online** until a renewed paper is approved. Overdue service only warns; it never blocks.

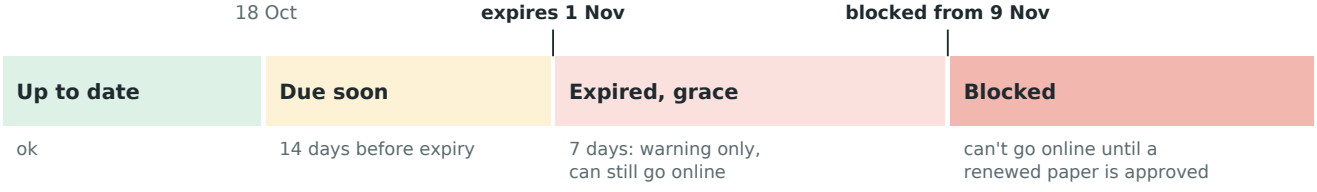
The items

type	label (show this)	Kind	When it is due (default)
`service`	Routine service	service record	Every 5,000 km or 3 months after the last approved service, whichever comes first. Needs an odometer reading.
`tyres_brakes`	Tyres & brakes check	service record	Every 6 months.
`battery_health`	Battery health check	service record	Every 12 months. Electric and plug-in hybrid cars only.
`insurance`	Insurance	legal paper	Expiry date of the approved insurance certificate.
`roadworthiness`	Roadworthiness certificate	legal paper	Expiry date of the approved certificate.
`vehicle_licence`	Vehicle licence	legal paper	Expiry date of the approved licence.

Item statuses

status	status_label	Meaning	Suggested colour
`ok`	Up to date	Nothing to do.	green
`due_soon`	Due soon	Within 14 days of the due date, or within 500 km of the due reading.	amber
`overdue`	Overdue	Service item past its date or distance. Warning only.	red
`expired`	Expired	Legal paper past its expiry. Grace period first, then blocks driving.	red (darker once `blocks_driving`)
`unknown`	No record yet	No approved service date, or no approved paper with an expiry date.	grey, with a call to action

Expired papers: grace period, then blocked



Example with the default 7-day grace: insurance expires on 1 Nov. From 2 Nov the item is `expired` and its message says “upload the renewed paper by 8 Nov 2026 to keep driving” (`blocks_driving_from:`

`"2026-11-09"`). From 9 Nov `blocks_driving` is true, `can_drive` is false, and going online is refused.

When a car becomes blocked: the driver gets a “You Can't Go Online” push, is taken offline (a driver mid-trip finishes the trip first), and loses the scheduled pickups they had reserved. For fleet cars, Operations get an alert. **When a renewed paper is approved**, the block lifts by itself and the driver gets “You Can Go Online Again”.

Odometer

- The driver (or a service record) gives a reading. Between readings the system adds completed trip distance, so `estimated_odometer_km` keeps moving.
- A reading can't be lower than an earlier one or higher than a later one. The 422 message explains which, e.g. “The reading is lower than the last one (48200 km). Check the number on the dashboard.”

Reminders

- Checked every day at 06:00 Lagos time, and whenever a car's status screen is opened.
- The driver gets one push per car, listing only the items whose status changed since the last message (no daily repeats).
- Operations get one daily digest of fleet items that are overdue or expired.

> **Not client-confirmed.** Intervals, the 14-day / 500 km reminder lead and the 7-day grace period are engineering recommendations (16 Oct 2026). The client has not confirmed them. Operations can change them all on the settings screen (Screen E).

B2. Driver app screens

Screen 1 — Vehicle health

`GET` `/driver/vehicle/status` driver

Field	Notes
`vehicle`	`{id, plate_number, make, model, fuel_type, is_fleet, shift_mode, odometer_km, odometer_read_at, estimated_odometer_km}`
`can_drive`	false when blocked. Show a full-width red banner with `block_reason`.
`block_reason`	e.g. “Insurance expired on 1 Nov 2026.”
`records_managed_by`	`driver` (own car: show “Log service” buttons) or `operations` (fleet car: hide them; show “E-tiGo services this car”).
`items[]`	The checklist. See the item fields below.
`charging`	EV block (Part A, Screen 1); null for cars that can't charge.
`pending_records[]`	Records the driver submitted that Operations haven't checked yet. Show as “Awaiting review”.

Item fields

Field	Type	Notes
`id, type, label`	string	Show `label`.
`is_legal_paper`	bool	true → call to action is “Upload renewed paper”; false → “Log service”.
`status, status_label`	string	See item statuses.
`message`	string	Ready-to-show sentence, e.g. “Routine service is due by 12 Dec 2026 or at 53200 km.” Use it as the row subtitle.
`due_at`	date | null	Due or expiry date.
`days_until_due`	int | null	Negative = days past. “in 9 days” / “3 days ago”.

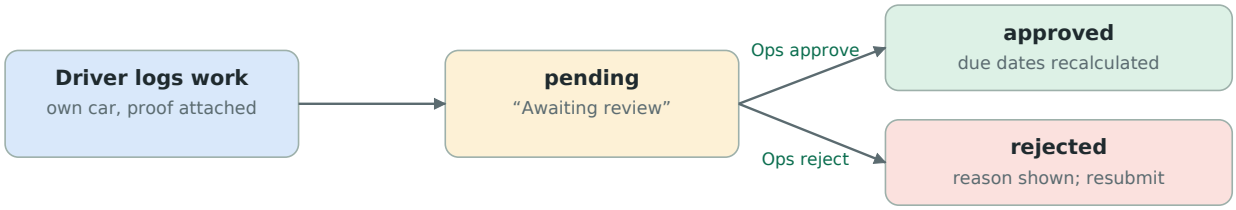
Field	Type	Notes
`due_odometer_km`, `km_until_due`	int | null	Service only. “in 850 km”.
`last_done_at`, `last_odometer_km`	date / int | null	Last approved service, or when the paper was approved.
`blocks_driving`	bool	This item is keeping the car off the road.
`blocks_driving_from`	date | null	Expired paper still in its grace period: the day driving stops. Show a countdown.

Screen 2 — Log maintenance (own car only)

POST `/driver/vehicle/maintenance-records` driver, multipart, max 10 per minute

Field	Required	Rules
`item_type`	yes	`service` | `tyres_brakes` | `battery_health` (battery only for plug-in cars). Papers are not logged here.
`performed_at`	yes	Date, not in the future, within the last 2 years.
`odometer_km`	for service	Integer 0–2,000,000. Required for Routine service. Must fit between earlier and later readings.
`workshop`	no	Max 120 characters.
`notes`	no	Max 1000.
`driver_repair_id`	no	Link to one of the driver's repair requests (Part C), e.g. “This was for request RPR-0042”.
`proof`	yes	Receipt or workshop report: jpg, png or pdf, max 10 MB. Camera or file picker.

201: `{message: "Record submitted. E-tiGo will check it shortly.", record}` with status `pending`. **403** on fleet cars: “Fleet cars are serviced and recorded by E-tiGo operations. Contact fleet support.”



Screen 3 — Maintenance history

GET `/driver/vehicle/maintenance-records?page=` driver, 20 per page, newest first

Record field	Notes
`id`, `vehicle_id`	
`item_type`, `item_label`	Show `item_label`.
`performed_at`, `odometer_km`, `workshop`, `notes`	
`proof_url`	Signed link to the receipt (image or PDF). Can be null for Operations entries.
`driver_repair_id`	Linked repair request, if any.
`status`, `status_label`	`pending` Awaiting review, `approved` Approved, `rejected` Rejected.
`rejection_reason`	Show on rejected records, with a “Submit again” button.
`reviewed_at`, `created_at`	timestamps

Screen 4 — Update odometer

`POST` `/driver/vehicle/odometer` driver, body {odometer_km}

Returns `{message: "Odometer reading saved.", odometer_km, odometer_read_at}`. Prompt for it on the health screen when `odometer_read_at` is old, since the km-based service date depends on it.

Screen 5 — Upload a renewed paper

`POST` `/driver/documents` driver, multipart (existing onboarding endpoint)

Fields: `type` (`insurance_certificate | roadworthiness_certificate | vehicle_licence`), `document` (file), `expires_at` (date after today; required for these papers). The paper goes to Operations' existing driver-document review; once approved, the item updates and any block lifts. Fleet cars get 422: “Your fleet car's papers are managed by E-tiGo operations.”

Screen 6 — Going online

`POST` `/driver/toggle-online` driver

- **Blocked:** 422 `{message: "Cannot go online.", reasons: [...]}`. One reason reads e.g. “Insurance expired on 1 Nov 2026. Upload the renewed paper; you can go online once E-tiGo approves it.” Show the list, with an upload button.
- **Allowed with a warning:** 200 with `vehicle_notice` set when papers are expiring soon or in grace. Show it as a dismissible banner.

Driver notifications

Category `compliance`. `data.vehicle_id` is included where relevant; deep-link to the health screen.

data.type	Title	Sent when
`vehicle_maintenance_due`	Vehicle Check Due	Daily check: items changed to due soon, overdue, expired or still missing. Body joins the items' messages.
`vehicle_blocked`	You Can't Go Online	The grace period ended. Body: block reason + “Upload the renewed paper...”
`vehicle_unblocked`	You Can Go Online Again	A renewed paper was approved.
`maintenance_record_reviewed`	Maintenance Record Checked	Operations approved or rejected a record (rejection includes the reason).

B3. Admin dashboard screens (Operations)

Screen A — Maintenance overview (what needs attention)

`GET` `/admin/vehicle-maintenance` Operations, 30 per page

Query param	Notes
`status`	Comma list of item statuses. Default `due_soon,overdue,expired`.
`type`	One item type, e.g. `insurance`.
`fleet_only`, `blocked_only`	true/false toggles.
`search`	Plate, make or model.

Response: `items[]` (item fields from B2 plus `vehicle: {id, plate_number, make, model, is_fleet, is_blocked, driver: {id, name}}`), `summary` and `meta`. Sorted expired, overdue, due soon, unknown, then by due date. `summary` has a count per item status present (`ok, due_soon, overdue, expired, unknown`), plus `blocked_vehicles` and `pending_records`. Use them as KPI tiles; a missing status key means 0.

Screen B — Records to review

`GET` `/admin/vehicle-maintenance/records?status=pending&vehicle_id=` Operations, oldest first

`POST` `/admin/vehicle-maintenance/records/{id}/review` body {action: approve|reject, rejection_reason}

Records include, for admins, `vehicle: {id, plate_number, is_fleet}` and `submitted_by: {id, name, type}`. Show the proof (`proof_url`) next to the details so the reviewer can compare. `rejection_reason` is required to reject (max 500) and is sent to the driver. 422 if the record was already reviewed, or if its odometer reading conflicts with another one.

Screen C — Vehicle maintenance detail

`GET` `/admin/vehicles/{vehicle_id}/maintenance` Operations

Returns `vehicle` (full vehicle incl. `odometer_km, estimated_odometer_km, battery_level_percent, is_blocked, block_reason, is_fleet, shift_mode, fuel_type`), `items[]`, `records[]` (latest 50, with submitter) and `documents[]` (the car's papers). Suggested layout: header with plate, driver and block state; checklist; history tabs for records and papers.

Action: record work (fleet cars, or on a driver's behalf)

`POST` `/admin/vehicles/{vehicle_id}/maintenance-records` Operations, multipart

Same fields as the driver form (Screen 2), but `proof` is optional and the record is **approved at once**. Returns `{message: "Maintenance recorded.", record, items}`. Use `items` to refresh the checklist.

Action: upload a paper

`POST` `/admin/vehicles/{vehicle_id}/documents` Operations, multipart

Fields `type`, `document`, `expires_at` (required for insurance, roadworthiness and vehicle licence). Approved at once and replaces the car's earlier paper of that type. Returns `{message, document, items}`.

Screen D — Unlinked papers

`GET` `/admin/vehicle-documents/unlinked` Operations

`POST` `/admin/vehicle-documents/{document_id}/link` body {vehicle_id}

Vehicle papers that couldn't be matched to one car when maintenance launched (the driver had no car, or several). Each comes with `driver: {id, name}`. A small clean-up screen: pick the vehicle, link.

Screen E — Maintenance settings

`GET` `/admin/settings/vehicle-maintenance` Operations

`PUT` `/admin/settings/vehicle-maintenance` send only changed fields

Setting	Default	Rules
`enabled`	true	Off: items are still tracked and shown, but no reminders or blocks.
`intervals`	service 5000 km / 3 mo; tyres_brakes 6 mo; battery_health 12 mo	`{service: {km, months}, ...}`. km 500–100,000, months 1–60; each item needs at least one.
`fuel_type_intervals`	none	Overrides per fuel type, e.g. `{electric: {service: {km: 10000, months: 6}}}`.
`due_soon_days`	14	0–90
`due_soon_km`	500	0–5000
`legal_grace_days`	7	0–60

Setting	Default	Rules
block_on_expired_papers	true	Turn the block off and keep only the warnings.

B4. Example responses

GET /driver/vehicle/status (trimmed)


```
{
  "vehicle": {
    "id": "v1a2...", "plate_number": "LND-482-KJ", "make": "BYD", "model": "Atto 3",
    "fuel_type": "electric", "is_fleet": false, "shift_mode": false,
    "odometer_km": 48200, "odometer_read_at": "2026-10-01T08:00:00+00:00", "estimated_odometer_km": 52350
  },
  "can_drive": true,
  "block_reason": null,
  "records_managed_by": "driver",
  "items": [
    {"id": "i1...", "type": "service", "label": "Routine service", "is_legal_paper": false,
     "status": "due_soon", "status_label": "Due soon", "due_at": "2026-11-20", "days_until_due": 23,
     "due_odometer_km": 53200, "km_until_due": 850, "last_done_at": "2026-08-20", "last_odometer_km": 48200,
     "blocks_driving": false, "blocks_driving_from": null,
     "message": "Routine service is due by 20 Nov 2026 or at 53200 km."},
    {"id": "i2...", "type": "insurance", "label": "Insurance", "is_legal_paper": true,
     "status": "expired", "status_label": "Expired", "due_at": "2026-10-25", "days_until_due": -3,
     "due_odometer_km": null, "km_until_due": null, "last_done_at": "2025-10-26", "last_odometer_km": null,
     "blocks_driving": false, "blocks_driving_from": "2026-11-02",
     "message": "Insurance expired on 25 Oct 2026; upload the renewed paper by 1 Nov 2026 to keep
      driving."},
    {"id": "i3...", "type": "tyres_brakes", "label": "Tyres & brakes check", "is_legal_paper": false,
     "status": "unknown", "status_label": "No record yet", "due_at": null, "days_until_due": null,
     "...": "...", "message": "Add the date of your last tyres & brakes check."}
  ],
  "charging": {
    "battery_level_percent": 42, "battery_level_at": "2026-10-28T07:15:00+00:00",
    "active_reservation": {"id": "a1f0...", "status": "confirmed",
                           "station": {"id": "9d4c...", "name": "Lekki Phase 1 Hub"}, "confirmed_slot":
      "02:30 pm"},
    "reservation_fee": {"amount_kobo": 50000, "waived": false, "waiver_reason": null}
  },
  "pending_records": [
    {"id": "r9...", "item_type": "tyres_brakes", "item_label": "Tyres & brakes check", "performed_at":
      "2026-10-26",
     "status": "pending", "status_label": "Awaiting review", "proof_url": "https://...", "...": "..."}
  ]
}
```


GET /admin/vehicle-maintenance (trimmed)


```
{
  "items": [
    {"type": "insurance", "label": "Insurance", "status": "expired", "status_label": "Expired",
     "due_at": "2026-10-19", "days_until_due": -9, "blocks_driving": true,
     "message": "Insurance expired on 19 Oct 2026; you can't go online until a renewed paper is approved.",
     "...": "other item fields",
     "vehicle": {"id": "v7...", "plate_number": "KJA-210-XY", "make": "Toyota", "model": "Corolla",
                 "is_fleet": true, "is_blocked": true, "driver": {"id": "d3...", "name": "Musa Ibrahim"}}}
  ],
  "summary": {"ok": 212, "due_soon": 14, "overdue": 6, "expired": 3, "unknown": 41,
              "blocked_vehicles": 2, "pending_records": 5},
  "meta": {"current_page": 1, "last_page": 1, "per_page": 30, "total": 23}
}
```

Part C — Repair & service requests

Drivers can already ask E-tiGo for a repair or a maintenance visit from the app. This is separate from the maintenance checklist in Part B.

C1. Driver side (built)

`GET` `/driver/repairs` driver

`POST` `/driver/repairs` driver, multipart

`GET` `/driver/repairs/{id}` driver

`POST` `/driver/repairs/{id}/attachments` driver, multipart {attachment, title}

`DELETE` `/driver/repairs/{id}/attachments/{attachment_id}` driver

Form field	Required	Rules
`type`	yes	`repair` or `maintenance`
`title`	yes	Max 150, e.g. “Brake noise”.
`details`	yes	Max 2000.
`preferred_date`	yes	Date.
`preferred_time`	yes	Text, max 50.
`attachment`	no	Photo or PDF (jpg, png, webp, pdf), max 10 MB.

Response object: `id, reference_number, type{value,label}, status{value,label}, title, details, preferred_date, preferred_time, summary_note, vehicle_name, vehicle_plate, vehicle_color, workshop_visit{scheduled_time, instructions, support_phone}, appointment, cancellation_reason, cancelled_by, progress_steps[{title, subtitle, state}], attachments[], submitted_at`. Status values: `in_progress, scheduled, completed, cancelled`. Step `state`: `completed | in_progress | upcoming` (draw as a stepper).

`POST` `/driver/repairs/{id}/cancel` driver, body {reason?}

New: the driver can cancel their own request while it is open (`can_cancel: true`). Files E-tiGo adds (`uploaded_by: operations`) can't be removed by the driver; hide the delete icon on them.

Status	label	Driver sees
`in_progress`	In progress	Request received; E-tiGo is arranging a technician. Can cancel.
`scheduled`	Scheduled	Workshop visit booked: `appointment` (date, time, workshop, address, technician, contact phone, instructions). Can cancel.
`completed`	Completed	`completion_summary`, and the workshop report in `attachments`.
`cancelled`	Cancelled	`cancellation_reason`; `cancelled_by` is `driver` or `operations`.

data.type	Title	Sent when
`repair_request_scheduled`	Workshop Visit Booked	Operations booked or rebooked the visit. Body has the time and workshop.
`repair_request_completed`	Vehicle Service Complete	Operations completed it. Body has the summary.
`repair_request_cancelled`	Service Request Cancelled	Operations cancelled it. Body has the reason.

C2. Admin dashboard screens (Operations)

Screen A — Repair requests queue

`GET` `/admin/repair-requests` Operations

Query param	Notes
`status`	Comma list. Default `in_progress,scheduled` (the open ones). Tabs: New | Booked | Completed | Cancelled.
`type`	`repair` or `maintenance`.
`search`	Reference (REP-1041), title, plate, driver name or phone.
`from, to`	Submitted between these Lagos dates.
`vehicle_id`	One car (link from the vehicle maintenance screen).

Response: `repairs[]`, `summary` (count per status, for tab badges) and `meta`. Order: new requests oldest first, then booked visits soonest first, then the rest. Suggested columns: reference, driver (`driver.name`, `driver.phone`), vehicle plate, type, title, preferred date and time, status, booked time (`appointment.date`, `appointment.time`), handled by (`handled_by.name`).

Screen B — Request detail

`GET` `/admin/repair-requests/{id}` Operations

Header: reference, status, driver, vehicle. Body: title and details, preferred time, photos (`attachments`), the progress stepper, appointment and completion summary. Actions depend on status: open requests offer Book visit, Complete and Cancel; every status offers Add file.

Action: Book (or rebook) the workshop visit

`POST` `/admin/repair-requests/{id}/schedule` Operations

Field	Required	Notes
`scheduled_at`	yes	Date + time, ISO 8601 with the Lagos offset. Not in the past. Prefill from the driver's preferred date.
`workshop_name`	yes	Max 150.
`workshop_address`	yes	Max 300.
`technician_name`	no	Max 100. Shown on the “Technician assigned” step.
`contact_phone`	no	Max 30. Replaces the fleet support number for the driver.
`instructions`	no	Max 1000, e.g. “Ask for Emeka at the front desk.”

Action: Complete

`POST` `/admin/repair-requests/{id}/complete` Operations, multipart

`summary` is required (5–2000), shown to the driver: what was done and anything they should know. `report` is an optional file (jpg, png, webp or pdf, up to 10 MB). Tip: offer “Also log this on the car's maintenance checklist”, which calls `POST /admin/vehicles/{vehicle_id}/maintenance-records` with `driver_repair_id`.

Action: Cancel

`POST` `/admin/repair-requests/{id}/cancel` body {reason}

`reason` is required (5–500) and shown to the driver.

Action: Add file

`POST` `/admin/repair-requests/{id}/attachments` multipart {attachment, title?}

A quote, photos or a report, at any stage. Refused actions return 422 with `current_status`, as on the EV screens.

> **Not client-confirmed.** The repair statuses, notifications and the rebook rule are engineering picks (28 Oct 2026), not client-confirmed.

Part D — What changed in revision 2

The gaps found in revision 1, and how each was closed. Nothing here breaks a screen built on revision 1.

#	Revision 1 gap	Now
1	Repair requests had no admin side	Built: Operations queue, detail, book, complete, cancel, add file; driver can cancel (Part C).
2	Admins couldn't end a driver's EV reservation	Built: Cancel (with optional fee refund) and End session (Part A, Screen F).
3	No admin list of all EV reservations	Built: `GET /admin/ev-reservations` with filters and search (Screen F).
4	Request form had no “how long”	Added `estimated_charge_minutes` to the request form and to Start charging.
5	Queue-ahead fields always null	`queue_ahead_count` and `queue_ahead_description` now filled while queued.
6	Mixed money units	Added `fee_amount_kobo`; `fee_amount` is unchanged.
7	Different response keys	Confirm and decline now return the object under `reservation` too; `charge_request` kept.
8	Display-only times	Added `check_in_deadline` (timestamp).
9	No live updates in the driver app	Added `ev.reservation.updated` on `driver.{userId}`.
10	— (new client rule)	Drivers still paying off an E-tiGo car charge free: `fee_waiver_reason: hire_purchase`; `charging.reservation_fee` on vehicle status.
11	Values only in server config	EV settings screen (Screen G); `ev_charging` and `vehicle_maintenance` in `GET /config`; refund option on End.

Values awaiting client confirmation

These are engineering defaults. Every one is now a setting Operations can change without a release (EV: Screen G; maintenance: Part B, Screen E), and the apps read them from `GET /config`, so a client decision needs no app or backend change. They have gone to the client in `doc/client/EV_MAINTENANCE_REPAIRS_DECISIONS.pdf` (EV-1 to EV-7, VM-1 to VM-3, RP-1).

Value	Default	Where it changes
Reservation fee	₦500	EV settings `reservation_fee`
Paused hire-purchase agreement counts as paying	yes	EV settings `waive_fee_while_agreement_paused`
Stall hold / check-in	30 min	EV settings `reservation_expiry_minutes`
Free cancellation	2 min	EV settings `free_cancellation_minutes`
Queue size, queue window, retry wait	10 / 15 min / 15 min	EV settings
Refund when Operations cancel or end	Operations' choice each time (off by default)	`refund_fee` on Cancel and End
Service intervals	5,000 km or 3 mo; 6 mo; 12 mo	Maintenance settings `intervals`
Reminder lead	14 days / 500 km	Maintenance settings
Grace before blocking	7 days	Maintenance settings `legal_grace_days`
Rebook a workshop visit	allowed while open	— (product rule)